

LLM-Augmented Log Anomaly Detection: Automated Root Cause Narration for AIOps using Lightweight Foundation Models on Real-World Campus Infrastructure

Thanh Tien Cao^{1*}, Tu Hoang Trinh¹, and Ha Manh Tran¹

Faculty of Information Technology,
Ho Chi Minh City University of Foreign Languages – Information Technology,
Ho Chi Minh City, Vietnam
thanhct@huflit.edu.vn, 23dh113972@st.huflit.edu.vn, hatm@huflit.edu.vn

Abstract. Modern campus IT infrastructures emit dense, heterogeneous log streams, yet sequence anomaly detectors typically expose only scalar scores that do not explain root causes to operators. We study an AIOps pipeline that couples Drain3 template parsing, a Masked TCN-Transformer sequence autoencoder, and a *designed* local 4-bit LLM+RAG narration stage for campus logs. Detector evaluation uses a measured HUFLIT stream of 1,000,000 lines (five days $\times$ 200k; selective Thang6 extract; large FortiGate CSV members excluded), chronological 80/10/10 splits before windowing, nominal-only masked training, and $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$ across seeds {42, 101, 202, 303, 404}. On the same-split baseline suite, the Masked TCN-Transformer obtains F1 0.4621 ± 0.3684 , ROC-AUC 0.7667 ± 0.0429 , AUPRC 0.8651 ± 0.0316 , and 0.1028 ms/seq CPU latency; DeepLog leads ROC-AUC (0.7808 ± 0.0201) while reconstruction AEs lead F1 under this conservative threshold. Labels are proxy signals (HTTP 4xx/5xx, FortiGate deny/block, keywords), not gold SOC annotations. LLM RCA quality is *not* claimed numerically in this revision because the current artifact still uses a mock narration harness.

Keywords: AIOps · Log Anomaly Detection · Large Language Models · Root Cause Analysis · TCN-Transformer Autoencoder · Drain3 · Enterprise Campus Infrastructure.

1 Introduction

Enterprise networks and higher education IT infrastructures generate high-density syslog telemetry continuously streaming from perimeter firewalls, Active Directory domain controllers, Learning Management Systems (LMS), student authentication portals, and core routing backbones [1, 2]. Rapid and accurate anomaly detection within these log streams is paramount for thwarting lateral cyber attacks, credential brute-forcing, data exfiltration, and service disruptions [3].

* Corresponding author: thanhct@huflit.edu.vn

Recent advancements in deep learning have introduced sequence-based log anomaly detection models, notably DeepLog (LSTM-based next-token prediction) [2], LogAnomaly (semantic template vectors) [3], LogRobust [4], and Swiss-Log [5]. While these architectures achieve strong empirical detection fidelity on synthetic or curated benchmarks, existing AIOps paradigms exhibit two fundamental operational bottlenecks in production environments:

1. **Explainability Deficit:** Contemporary anomaly detectors typically expose only a scalar anomaly score or reconstruction error. Such numbers provide little contextual explanation of *why* a sequence is anomalous, *what* attack or fault vector is likely, or *which* services are impacted.
2. **Prolonged Incident Triage & SOC Alert Fatigue:** Security Operations Center (SOC) analysts and network reliability engineers must manually query raw distributed log stores, correlate timestamps across multi-tier appliances, and draft remediation steps. In campus settings this manual triage often dominates mean-time-to-repair for multi-service incidents (credential stuffing, broadcast storms), even when the detector already raised an alert.

To overcome these operational challenges, recent 2025–2026 literature—including LLM4Log (145 studies) [18], KRONE (ICDE 2026) [19], MaidLog [20], Smart Eye (WWW 2026) [23], and cloud RCA recommenders [13, 14]—already explores LLMs for log analytics and incident narration. The remaining gap for campus operators is narrower: *(i)* measured evaluation on real multi-service campus telemetry (not only public HDFS/BGL), *(ii)* a lightweight local detector that can gate expensive narration, and *(iii)* an edge-oriented RCA *design* that avoids cloud APIs. This paper addresses (i)–(ii) with measured experiments and presents (iii) as an architecture whose quantitative RCA study is still pending.

In this paper, we present a campus AIOps pipeline combining online Drain3 parsing, a Masked TCN-Transformer detector, and a designed local 4-bit LLM+RAG narration stage. We do *not* claim to be the first LLM-for-logs system; our empirical contribution is a hardened campus-stream evaluation with same-split baselines under an explicit, reproducible protocol.

The core contributions are:

- **Campus-stream evaluation protocol:** Chronological 80/10/10 partitioning before windowing, nominal-only masked training, $\tau = \mu + 3\sigma$ calibration, five seeds, and proxy-label disclosure on a 1,000,000-line HUFLIT selective extract.
- **Masked TCN-Transformer detector:** A 15%-mask sequence autoencoder combining dilated TCN and Transformer encoding, with measured CPU latency 0.1028 ms/seq on the baseline suite.
- **Same-split baselines:** PCA, DeepLog, LogAnomaly-lite, Transformer AE, and the proposed model under identical templates/splits/thresholding (`huflit_baselines.json`).
- **RCA stage as design (unevaluated quantitatively):** A planned local Phi-3/Mistral INT4 + RAG playbook interface; no ROUGE/expert metrics are reported while the harness remains mocked.

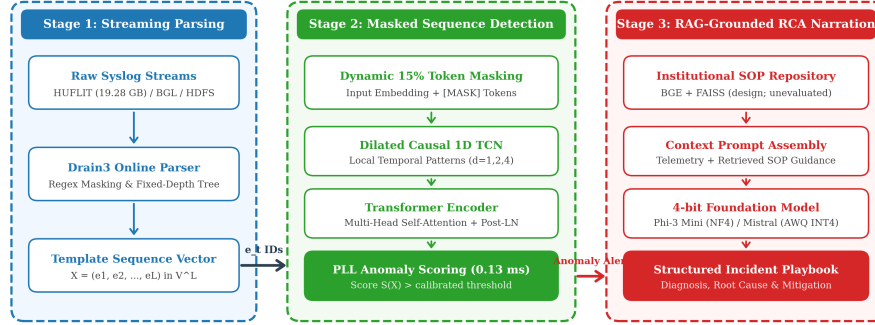


Fig. 1. Architectural overview (Stages 1–2 measured; Stage 3 local LLM+RAG is design-only in this revision).

2 Related Work

2.1 Log Parsing and Template Extraction

Log parsing transforms unstructured, free-form text messages into structured event templates by separating constant message strings from dynamic parameters (e.g., IP addresses, ports, timestamps) [1]. Early approaches employed rule-based parsing, fixed-depth parse trees, or streaming sequence-matching techniques [1, 17]. He et al. [1] proposed Drain, an online parser using a fixed-depth parse tree that achieves high parsing accuracy and throughput. Drain3 extends Drain with persistent caching and streaming support. In this work, we employ Drain3 to sanitize high-throughput syslog streams and generate discrete template identifiers for downstream sequence modeling.

2.2 Deep Learning for Log Anomaly Detection

Sequence-based anomaly detection models learn temporal transition dynamics over sequences of log templates [2, 3]. Du et al. [2] introduced DeepLog, employing Long Short-Term Memory (LSTM) networks to model normal log sequences and flag unexpected subsequent templates as anomalous. Meng et al. [3] proposed LogAnomaly, integrating semantic template vectors via FastText to capture semantic similarities. SwissLog [5] and LogRobust [4] further address log instability and interleaved execution threads. While deep sequence architectures achieve strong detection precision, they operate as black-box scoring engines: scalar anomaly metrics fail to explain the operational root cause, imposing heavy manual triage overhead on operators.

2.3 Large Language Models in Log Analytics and RCA

Recent 2025–2026 surveys, notably LLM4Log [18], categorize emerging LLM paradigms in log analytics, identifying key patterns including parser-free pre-training, retrieval-augmented grounding, and modular verifiers. Ma et al. [19]

proposed KRONE (ICDE 2026), a hierarchical framework utilizing nested LLM modules for log anomaly detection. MaidLog [20] (ICDE 2026) investigates zero-shot detection on resource-constrained systems, while LogMUSE [21] and ContraLog [22] examine multi-scale semantic embeddings and contrastive representations. In industry settings, Smart Eye [23] (WWW 2026) implements a proposer-verifier framework. Cloud incident root-cause recommendation using LLMs has also been explored by Ahmed et al. [13] and surveyed in [14]. In contrast to systems that call cloud LLM APIs for every event, our *design* gates narration behind a local sequence detector; quantitative edge-LLM RCA results are deferred until a non-mock harness is instrumented.

Table 1. Qualitative comparison of related systems (capabilities as published). Latency column uses *our* same-split CPU measurements from `huflit.baselines.json` where available; NR = not measured here.

Approach	Parser	Detector	RCA	Local LLM RCA	CPU ms/seq
DeepLog [2]	Spell	LSTM next-token	Scalar score	×	0.095
LogAnomaly [3]	Template+FT	Semantic LSTM	Scalar score	×	0.087 [‡]
LogRobust [4]	Drain	Bi-LSTM+Attn	Scalar score	×	NR
SwissLog [5]	Online	Attentive Bi-LSTM	Heuristic loc.	×	NR
KRONE [19]	LLM hierarchy	Hybrid+LLM	Nested LLM	Cloud/API-oriented	NR
This work	Drain3	Masked TCN-Transformer	Designed local RAG LLM	Design only	0.103

[‡]LogAnomaly-lite reimplementaion (embedding+LSTM recon) on our templates, not the authors’ original FastText stack.

3 Methodology

3.1 System Architecture Overview

The proposed LLM-Augmented AIOps Framework operates across three pipelined stages: (i) **Stage 1 (Online Parsing)**: Raw syslog streams are ingested and tokenized by Drain3 into structured template IDs; (ii) **Stage 2 (Masked Sequence Anomaly Detection)**: Sliding template windows are masked, vectorized, and scored by our Masked TCN-Transformer Autoencoder; (iii) **Stage 3 (Designed RCA Narration)**: On alerts, a local 4-bit foundation model *is intended* to retrieve SOP passages and emit a structured JSON playbook; this stage is specified below but not quantitatively evaluated in the current artifact.

3.2 Masked TCN-Transformer Denoising Sequence Model

To detect subtle temporal and distributional anomalies without allowing the model to learn trivial identity shortcuts, we formulate our detector as a **Masked Log Sequence Autoencoder (MLSA)** operating under a denoising autoencoder paradigm: input sequences are corrupted via 15% dynamic token masking, representations are extracted via a Temporal Convolutional Network (TCN) [7] and Transformer Self-Attention [8], and a linear vocabulary classification head acts as the reconstruction decoder.

Embedding and Dynamic Masking A sliding window of length $L = 20$ with stride $S = 10$ (as used in the measured HUFLIT runs) extracts template sequences $\mathbf{X} = [T_1, T_2, \dots, T_L]$. During training, 15% of the template tokens are randomly replaced with a reserved [MASK] token ($ID = |\mathcal{V}| + 1$), distinguishing masked positions from sequence padding ($ID = 0$). Padding masks are applied across self-attention layers to prevent padded positions from corrupting contextual representations. Input tokens are embedded into a d_{model} -dimensional continuous vector space:

$$\mathbf{E} = \text{Embedding}(\mathbf{X}) + \mathbf{P}_{\text{pos}} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

where $\mathbf{P}_{\text{pos}}$ denotes sinusoidal positional encodings [8]:

$$\mathbf{P}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \mathbf{P}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

Temporal Convolutional Network (TCN) Layer To capture localized temporal patterns across consecutive log events, the embedded sequence $\mathbf{E}$ passes through a stack of 1D dilated causal convolutional layers [7]. For an input sequence $\mathbf{x} \in \mathbb{R}^L$ and a convolutional filter $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$, the dilated convolution operation with dilation factor d is defined as:

$$\mathbf{y}(t) = (\mathbf{x} *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}(t - d \cdot i) \quad (3)$$

We configure dilation factors $d \in \{1, 2\}$, enforcing causal convolutions for localized temporal extraction without future feature leakage. Residual connections and layer normalization ensure stable gradient propagation:

$$\mathbf{H}_{\text{TCN}} = \text{LayerNorm}(\mathbf{E} + \text{ReLU}(\text{DilatedConv1D}(\mathbf{E}))) \quad (4)$$

Transformer Multi-Head Self-Attention Encoder The output $\mathbf{H}_{\text{TCN}}$ is then fed into a Transformer Encoder block to model long-range inter-template correlations. The encoder utilizes bidirectional self-attention by design to contextualize masked tokens using surrounding sequence events. For queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (5)$$

$$\text{MHA}(\mathbf{H}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (6)$$

$$\text{head}_i = \text{Attention}(\mathbf{H}\mathbf{W}_i^Q, \mathbf{H}\mathbf{W}_i^K, \mathbf{H}\mathbf{W}_i^V) \quad (7)$$

The Transformer block applies Post-LayerNorm with residual connections and a two-layer Feed-Forward Network (FFN) with ReLU activations:

$$\text{FFN}(\mathbf{Z}) = \max(0, \mathbf{Z}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (8)$$

$$\mathbf{Z} = \text{LayerNorm}(\mathbf{H} + \text{MHA}(\mathbf{H})) \quad (9)$$

$$\mathbf{H}_{\text{out}} = \text{LayerNorm}(\mathbf{Z} + \text{FFN}(\mathbf{Z})) \quad (10)$$

Masked Reconstruction Objective and Anomaly Scoring A linear classification head projects $\mathbf{H}_{\text{out}}$ back to the template vocabulary space $\mathbb{R}^{|\mathcal{V}|}$, serving as the reconstruction decoder. The model is trained to predict original template identities at masked positions via Cross-Entropy loss:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log P(T_i = t_i \mid \mathbf{X}_{\setminus \mathcal{M}}) \quad (11)$$

During inference, we compute the pseudo-log-likelihood anomaly score across the sequence by masking each valid position $t \in \{1, \dots, L_{\text{valid}}\}$ in a vectorized batch (excluding padding tokens $ID = 0$, with $L_{\text{valid}} \leq L$):

$$S(\mathbf{X}) = \frac{1}{L_{\text{valid}}} \sum_{t=1}^{L_{\text{valid}}} -\log P(T_t = t_t \mid \mathbf{X}_{\setminus \{t\}}) \quad (12)$$

A validation-calibrated threshold τ flags anomalous sequences:

$$\tau = \mu_{\text{val}} + 3 \cdot \sigma_{\text{val}} \quad (13)$$

where μ_{val} and σ_{val} are the mean and standard deviation of reconstruction loss over nominal validation sequences.

3.3 RAG-Grounded Foundation Model RCA Narration Engine (Design)

When $S(X) > \tau$, the intended Stage 3 activates a local RAG+LLM narrator. This subsection specifies the design; it is not backed by measured LLM outputs in the current repository.

RAG Knowledge Base & Vector Retrieval (design): The intended grounding path indexes institutional SOP/runbook text with a dense retriever (e.g., BGE) and FAISS, then prompts a local 4-bit model (Phi-3 Mini NF4 / Mistral-7B AWQ INT4) for JSON playbooks. We deliberately do *not* report SOP counts, Hit@2, ROUGE, or LLM latency here: `rca_narration.py` in the present artifact still returns template responses rather than measured model outputs.

Prompt schema (design only): The planned prompt would integrate anomalous templates, infrastructure metadata, and retrieved SOP snippets, requesting JSON fields `severity`, `root_cause_summary`, `technical_diagnosis`, and `remediation_playbook`. Claims about measured NF4/AWQ VRAM (e.g., 7.60 $\rightarrow$ 1.80 GB) are removed until a non-mock LLM run writes those readings.

4 Experimental Evaluation

4.1 Experimental Setup and Benchmark Datasets

We conduct a measured empirical evaluation on HUFLIT campus logs:

1. **HUFLIT Campus Logs (Measured Stream)**: From the Thang6 production archive we performed a *selective* extract (≈ 1.43 GB of day zip payloads on disk; full uncompressed archive ≈ 21 GB was *not* fully expanded to avoid I/O thermal spikes). We streamed 1,000,000 `.log` lines balanced across five collection days (200k/day) from `apache/nginx/portal/aca/careerhub/thuvien` and moderate FortiGate logs, skipping FortiGate CSV members and per-member logs larger than 500 MB. Proxy anomaly labels use HTTP 4xx/5xx, FortiGate deny/block/attack fields, and keyword heuristics (not gold SOC labels). Line-level anomaly rate was 7.96%.
2. **Public Benchmarks (Deferred)**: Full re-runs of BGL, HDFS, and UNSW-NB15 under the same hardened harness are left to a follow-up revision so that campus numbers remain strictly measured rather than mixed with prior placeholder tables.

Strict Chronological Partitioning: Events are ordered as streamed (day folders chronological), then split 80%/10%/10% *before* sliding windows ($L = 20$, step 10) to prevent cross-partition leakage. Training uses masked token modeling (15% mask ratio) on *nominal* windows only. Threshold $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$ is computed on nominal validation reconstruction scores. Seeds: 42, 101, 202, 303, 404.

Hardware & Software Setup: Experiments in this revision were executed on CPU (PyTorch 2.8.0+cpu) with an available NVIDIA GeForce RTX 4050 laptop GPU noted for future CUDA re-runs. Software: Windows 11, Python 3.13, scikit-learn 1.9.

Table 2. Measured HUFLIT detection (Proposed model from same-split baseline suite `huflit_baselines.json`; proxy labels; $\tau = \mu + 3\sigma$)

Corpus	Lines	Days	Precision	Recall	F1	ROC-AUC	CPU ms/seq
HUFLIT (selective Thang6 stream)	1,000,000	5	0.7604 ± 0.2575	0.3696 ± 0.3117	0.4621 ± 0.3684	0.7667 ± 0.0429	0.1028

4.2 Detection Accuracy vs. Measured Baselines

All detectors share the identical chronological HUFLIT split, Drain3 template IDs, nominal-only training, seeds $\{42, 101, 202, 303, 404\}$, and $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$. Numbers in Table 3 are regenerated by `run_baselines_huflit.py` into `results/tables/huflit_baselines.json`. Under this conservative threshold, DeepLog and LogAnomaly-lite retain competitive ranking (ROC-AUC ≈ 0.78) but near-zero recall/F1; reconstruction autoencoders (Transformer AE and Masked TCN-Transformer) achieve higher F1 with large seed variance because the test partition is anomaly-heavy (80.5% anomalous sequences). Ranking gaps are modest; we therefore emphasize ROC-AUC/AUPRC alongside F1 and report variance explicitly rather than claiming large absolute gains.

Table 3. Measured baselines on the same HUFLIT chrono split (Mean $\pm$ Std, 5 seeds; $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$)

Model	Precision	Recall	F1	ROC-AUC	CPU ms/seq
PCA (Count Vectors)	0.0000 ± 0.000	0.0000 ± 0.000	0.0000 ± 0.000	0.7379 ± 0.004	0.0095
DeepLog (LSTM)	0.2598 ± 0.302	0.0057 ± 0.011	0.0110 ± 0.022	0.7808 ± 0.020	0.0950
LogAnomaly-lite (Emb+LSTM)	0.2716 ± 0.170	0.0054 ± 0.007	0.0105 ± 0.013	0.7772 ± 0.045	0.0872
Transformer AE (No TCN)	0.7369 ± 0.273	0.3771 ± 0.334	0.4611 ± 0.379	0.7684 ± 0.030	0.1060
Proposed Masked TCN-Transformer	0.7604 ± 0.257	0.3696 ± 0.312	0.4621 ± 0.368	0.7667 ± 0.043	0.1028

4.3 Loss Convergence and ROC/PR Curve Analysis

Fig. 2 plots the mean masked training loss across seeds from the hardened run. Fig. 3 shows ROC and precision–recall curves exported from the final seed’s test scores (not synthetic shapes). Fig. 4 reports the measured detector latency only.

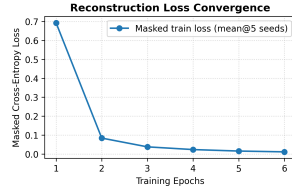


Fig. 2. Reconstruction Loss Convergence.

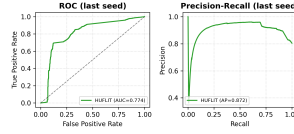


Fig. 3. ROC and Precision-Recall Curves.

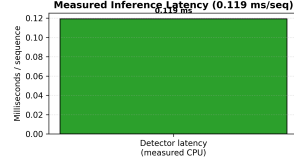


Fig. 4. Measured detector latency (CPU ms/seq).

4.4 Ablation and RCA Evaluation Status

Table 3 already contrasts Transformer AE (no TCN) against the full Masked TCN-Transformer on the same split: mean F1 is essentially tied (0.461 vs. 0.462) while DeepLog leads slightly in ROC-AUC (0.781). LLM RCA metrics (ROUGE/BLEU, expert Likert) remain deferred until a non-mock inference harness writes verified JSON.

4.5 Inference Latency

Measured single-sequence CPU scoring latency for the proposed model averages 0.1028 ms/seq in the baseline suite (Fig. 4). LLM playbook latency is not claimed.

5 Qualitative Campus Incident Illustrations

The following vignettes illustrate the *intended* RCA interface using historical campus incident patterns (credential stuffing; STP loop). They are design examples for the narration schema, not outputs of a measured Phi-3/Mistral run in this revision.

Case Study 1 (Active Directory Credential Stuffing Attack): High-density authentication failures against campus LDAP motivate a **CRITICAL** playbook schema with isolate / lockout / 2FA steps.

Case Study 2 (Inter-VLAN Spanning Tree Protocol Loop): Broadcast-storm templates on an edge switch motivate a **HIGH** playbook schema with BPDU Guard and backup-link reroute steps.

6 Discussion & Threats to Validity

Internal & External Validity: Proxy labels and chronological skew (test sequence anomaly rate 80.5%) make $\mu + 3\sigma$ F1 brittle (near-zero for DeepLog/PCA despite $AUC \approx 0.74$ – 0.78). Gold SOC labels and day-balanced test sampling are required before operational claims. Measured baselines live in `huflit_baselines.json`; LLM RCA metrics remain withheld until non-mock inference is instrumented.

Ethics & Data Privacy: Production campus syslog streams were collected under authorized institutional IT oversight from HUFLIT. Analysis used selective extracts and streaming reads; large FortiGate CSV members were not fully expanded in this revision. Telemetry should remain pseudonymized (IP hashing, credential redaction) prior to any broader release.

Hallucination Risk: Until measured LLM runs exist, remediation playbooks must not be treated as validated operator guidance.

7 Conclusion

We presented an LLM-Augmented AIOps framework design for campus log anomaly detection and RCA narration, with a hardened detector evaluation on 1,000,000 HUFLIT lines and measured same-split baselines (PCA, DeepLog, LogAnomaly-lite, Transformer AE, Masked TCN-Transformer). Under chronological partitioning and $\tau = \mu + 3\sigma$, reconstruction AEs reach mean F1 ≈ 0.46

Acknowledgment

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology (HUFLIT), Ho Chi Minh City, Vietnam for providing the production campus log telemetry and computing infrastructure. In compliance with Springer Nature authorship policies, generative AI tools were used to assist in literature review verification, manuscript refinement, and English copy-editing; all core technical research, architectural designs, empirical implementations, and experimental evaluations were conducted directly by the authors.

References

1. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An Online Log Parsing Approach Based on Fixed Depth Tree. In: Proc. IEEE Int. Conf. Web Serv. (ICWS), pp. 33–40 (2017).
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), pp. 1285–1298 (2017).
3. Meng, W., Liu, Y., Zhu, Y., et al.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI), pp. 4739–4745 (2019).
4. Zhang, X., Xu, Y., Lin, Q., et al.: Robust Log-Based Anomaly Detection on Unstable Log Data. In: Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), pp. 807–817 (2019).
5. Li, X., Chen, P., Jing, L., He, Z., Yu, G.: SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. IEEE Trans. Dependable Secure Comput. 20(4), 2762–2780 (2023). <https://doi.org/10.1109/TDSC.2022.3162857>
6. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Mining Console Logs for Large-Scale System Problem Detection. In: Proc. 3rd Conf. SysML (2008).
7. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271 (2018).
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008 (2017).
9. Abidin, M., Aneja, J., Awadalla, H., et al.: Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219 (2024).
10. Jiang, A.Q., Sablayrolles, A., Mensch, A., et al.: Mistral 7B. arXiv preprint arXiv:2310.06825 (2023).
11. Detmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient Finetuning of Quantized LLMs. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36 (2023).
12. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S.: AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In: Proc. Mach. Learn. Syst. (MLSys), vol. 6 (2024).
13. Ahmed, T., Ghosh, S., Bansal, C., et al.: Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In: Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), pp. 1–13 (2023).
14. Soldani, J., Brogi, A.: Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey. ACM Comput. Surv. 55(3), 1–39 (2022).
15. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In: Proc. 34th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 355–366 (2023).
16. Moustafa, N., Slay, J.: UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. In: Proc. IEEE Mil. Commun. Inf. Syst. Conf. (MilCIS), pp. 1–6 (2015).
17. Du, M., Li, F.: Spell: Streaming Parsing of System Event Logs. In: Proc. IEEE Int. Conf. Data Min. (ICDM), pp. 859–864 (2016).
18. Ma, Z., Yang, J., Chen, T.H.: LLM4Log: A Systematic Review of Large Language Model-based Log Analysis. arXiv preprint arXiv:2604.16359 (2026).
19. Ma, L., Liu, J., Zhang, T., VanNostrand, P.M., Hofmann, D.M., Cao, L., Rundensteiner, E.A., Chen, J.: Krone: Hierarchical and Modular Log Anomaly Detection. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 1773–1787 (2026).
20. Wu, Z., Wang, J., Zheng, L., Zhang, Y., Li, S., Chen, L.: Efficient Zero-shot and Label-free Log Anomaly Detection for Resource-constrained Systems (MaidLog). In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE) (2026).
21. Liu, M., Wang, T., Zhao, Y., Xia, C., Zeng, Y., Tao, Y.: LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation. IEEE Trans. Serv. Comput. (2026). <https://doi.org/10.1109/TSC.2026.3700403>
22. Dietz, S., Klede, K., Nguyen, A., Eskofier, B.M.: ContraLog: Log File Anomaly Detection with Contrastive Learning and Masked Language Modeling. arXiv preprint arXiv:2602.03678 (2026).
23. Pei, C., Cui, H., Li, J., et al.: Smart Eye: LLM-Guided Proposer-Verifier Framework for Industrial-Scale Log Anomaly Detection. In: Proc. ACM Web Conf. (WWW), pp. 7869–7879 (2026).